Nano-payments for supervised machine spending

PennyPilot

A spend-control card for AI finance agents buying paid financial facts through Circle Gateway x402, with receipts anchored on Arc.

- Arc proof live
- Gateway x402
- 50 tx proof

$0.003885

average paid action in the compact review

$0.25	$0.006	50
task budget	max per call	receipt tx

Live demo and Arc proof package

pennypilot-five.vercel.app

Agents need policy first

PennyPilot turns tiny provider calls into supervised, receipt-backed purchases.

Wallets are too blunt

An agent balance says what could be spent, not what should be spent for one finance task.

Facts need receipts

KYB, sanctions, invoice, risk, identity, and FX calls need provider accountability.

Sub-cent spend breaks easily

At a few thousandths of a dollar per action, payment overhead can erase the product margin.

PennyPilot

Payment first. Proof next.

Each paid fact has a purpose, price, provider, signature, and Arc receipt.

01 Set task policy

Budget, max per call, expiry, purpose, and provider allowlist.

02 Provider asks for payment

Paid fact endpoint returns x402 PAYMENT-REQUIRED.

03 Wallet signs

Browser creates a Gateway x402 PAYMENT-SIGNATURE.

04 Gateway settles

Provider verifies and settles the payment before returning data.

05 Provider signs receipt

Receipt hash and contract digest bind the call to the task.

06 Arc anchors proof

recordSpendWithProviderSignature records the receipt on the user's card.

Four layers, one audit trail

The app separates wallet funding, task policy, provider settlement, and Arc proof.

User-owned spend card

The wallet owns the card. A single task delegates bounded receipt anchoring to the PennyPilot relayer.

Provider API surface

Six paid providers expose the same challenge, payment, fact, and receipt contract.

UI	Connect wallet -> choose invoice task -> run paid review
x402	PAYMENT-REQUIRED -> PAYMENT-SIGNATURE -> Gateway settle
Contract	createStrictPolicyTask -> recordSpendWithProviderSignature
Verifier	bytecode, cap, allowlist, receipt replay, digest equality

Arc Testnet proof is already live

Explorer links and verifier commands make the receipt trail independently checkable.

Contract	0x4c8e...3047
Deploy tx	0x5bb0...a1bb
Policy task	task #1 / tx 0x86a4...b63b
Receipt tx	0x6006...f996
50 tx task	0x9f65...8ee7

Verifier-backed, not screenshot-only

The proof scripts read Arc Testnet and confirm contract bytecode, budget, per-call cap, provider allowlist, receipt replay status, provider spend, and digest equality.

- npm test
- verify:arc
- verify-50:arc

PennyPilot

The unit economics are the feature

These purchases are useful only when the payment rail can support frequent, tiny, policy-bound actions.

$0.023308

six-fact review spend

under the $0.25 policy budget

$0.003885

average paid action

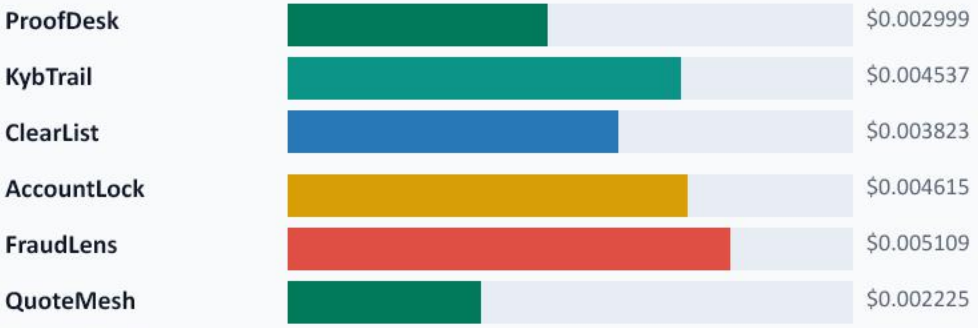
below the $0.01 threshold

50

Arc receipt tx

0.1805 USDC total proof spend

Provider prices stay inside the per-call cap



Agent spend policy

The same pattern works wherever an AI agent buys paid facts before a finance decision.

What the product proves

PennyPilot makes autonomous spending inspectable before it becomes operational risk: every paid fact is bounded, priced, signed, and traceable.

Invoice ops	validate duplicates, KYB, sanctions, delivery proof
Merchant risk	buy only the external checks needed for onboarding
Treasury ops	price FX/routing signals under a per-call cap
Audit trail	export receipts with signatures, hashes, and Arc anchors